UNIVERSITY OF PRINCE EDWARD ISLAND

Random Forests and Experimentation in Machine Learning

How forests learn. How experiments help us choose.

CS 4120 Machine Learning and Data Mining

From one decision tree to many

The previous lecture gave us readable rules. A forest combines many such models.

What one tree gives us

A sequence of feature tests.

A prediction at each leaf.

A small tree we can inspect.

Where one tree struggles

A deep tree can fit noise.

Small data changes can alter early splits and later predictions.

> **Combining different trees can reduce sensitivity to the training sample.**

The cycling example

Features describe the day. The target is the decision: Inside or Outside.

Day	Temperature	Rain	Wind	Decision
1	22	0	12	Outside
2	8	1	35	Inside
3	17	0	18	Outside
4	11	1	20	Inside

Tomorrow: 15 degrees, no rain, wind 30 km/h.

Four rows shown from the earlier lecture. A teaching example, not our benchmark.

Several trees contribute to one prediction

First intuition: ask several different trees to classify the same day.

Model	Predicted class
Tree 1	Inside
Tree 2	Outside
Tree 3	Inside

> **A simple majority vote would predict Inside: 2 of 3 trees agree.**

We will refine this intuition for scikit-learn on slide 9.

Two kinds of randomness

A random forest changes both the examples and the candidate split features.

Random rows

Each tree draws a bootstrap sample from the training rows.

Sampling with replacement allows repeated rows.

Random features

At each split, the tree considers a random subset of features.

The candidate subset changes across splits.

> **Different views of the same training data encourage different trees.**

A bootstrap sample can repeat a row

For this example, draw 8 times from 8 training rows, with replacement.

Original rows	One bootstrap sample
1 2 3 4 5 6 7 8	2 2 5 1 7 7 3 8
Every original row once	Rows 2 and 7 appear twice
8 available observations	Rows 4 and 6 are absent

> **Repeating a row changes its influence. It does not create new information.**

Feature sampling happens at each split

The tree still chooses a useful split among the features it considers.

Split	Candidate features	Example selected test
A	Temperature, Wind	Wind > 25?
B	Rain, Hour	Rain = 1?
C	Temperature, Hour	Temperature > 12?

> **A strong feature cannot dominate every split if it is sometimes unavailable.**

Illustrative splits. They are not a fitted model or measured result.

Diversity matters when models make mistakes

Combining identical predictions gives us no new evidence.

Actual class: Inside	Tree 1	Tree 2	Tree 3	Majority
Same error	Outside	Outside	Outside	Outside
Different errors	Outside	Inside	Inside	Inside

The forest works best when useful trees do not always fail on the same examples.

> **Averaging reduces variance most when the trees are less correlated.**

Class probabilities in scikit-learn

RandomForestClassifier averages each tree's class probabilities. [1]

Tree	P(Inside)	P(Outside)
1	0.80	0.20
2	0.40	0.60
3	0.90	0.10
Average	0.70	0.30

> **The largest average probability selects Inside.**

These illustrative probabilities are not automatically calibrated confidence levels.

A forest can also predict a number

For regression, the forest averages numeric predictions across trees.

Tree 1	Tree 2	Tree 3	Forest mean
18 minutes	24 minutes	21 minutes	21 minutes

(18 + 24 + 21) / 3 = 21

> **Our experiment focuses on classification. The same evaluation discipline applies.**

Tree depth and forest size do different jobs

More trees do not make each individual tree shallower.

Tree complexity

max_depth limits how far each tree grows.

min_samples_leaf requires more examples in each leaf.

Number of trees

n_estimators sets the ensemble size.

More trees often stabilize predictions, with diminishing gains and more computation.

> **A larger forest cannot repair data leakage or a poorly chosen target.**

The main hyperparameters

A hyperparameter controls training. The training process learns the actual splits.

Setting	What changes	Values in our experiment
max_depth	Maximum depth of a tree	3, 6, None
min_samples_leaf	Minimum rows per leaf	1, 4
n_estimators	Number of trees	100, 300
max_features	Features considered per split	"sqrt", 1.0

For 13 input features, "sqrt" considers 3. A float of 1.0 considers all 13.

None means no explicit depth limit. Other stopping rules still apply.

Out-of-bag observations

Some training observations do not appear in a particular tree's bootstrap sample.

A built-in validation idea

For a training row, gather predictions only from trees that did not train on that row.

These are its out-of-bag predictions.

Where it fits

With n draws from n rows, about 37% stay out on average for large n.

OOB is useful feedback for bootstrap forests.

> **We use the same CV folds for both models. OOB does not replace our final test.**

Decision Tree or Random Forest?

The choice depends on the evidence and the constraints of the task.

Consideration	Decision Tree	Random Forest
Explanation	One small tree is easy to trace	Many trees are harder to summarize
Stability	Can change sharply with data	Averaging often improves stability
Computation	Usually cheaper	More trees require more resources
Performance	Can be sufficient	Often strong on tabular data

Feature importance needs interpretation

Importance describes how a fitted model uses the available information. [6]

Impurity importance

Adds up split improvements within the fitted trees.

It can favor features with many possible split values.

Permutation importance

Shuffle a feature in validation data and measure the score decrease.

Correlated features can hide each other's contribution.

> **High importance is evidence of predictive use. It does not establish causation.**

What a forest does not guarantee

Common claim	More careful interpretation
"A forest cannot overfit"	It can learn noise or fail on a new population.
"More trees always improve accuracy"	Gains can level off. More trees also cost time.
"No preprocessing is needed"	Encode categorical inputs for these scikit-learn models.
"A high score proves the model is good"	The split and evaluation process must be credible.

> **Trees generally do not need feature scaling. Other preparation can still matter.**

Before choosing a forest

Discussion checkpoint

1. If every tree sees the same rows and features, how much diversity should we expect?

2. Would 1,000 trees fix labels that accidentally reveal the answer?

3. What evidence would justify choosing a forest over one small decision tree?

A fair model comparison

Example: classify wine cultivars from chemical measurements. [5]

Decision before training	Our experiment
Data	Wine dataset: 178 rows, 13 numeric features, 3 classes
Candidate models	Decision Tree and Random Forest
Selection metric	Macro F1: average the F1 score of each class equally
Comparison protocol	Same training rows, preprocessing, folds and metric

For each class: $F1 = 2PR / (P + R)$. P is precision and R is recall.

The small dataset supports a classroom demonstration, not a broad claim of superiority.

The test set has one job

A stratified 80/20 split creates development data and a final evaluation set.

142 training rows

Cross-validation compares candidate pipelines.

Grid search chooses a configuration.

The chosen pipeline refits on all 142 rows.

36 test rows

Set these aside at the beginning.

Use them once after the model and decision rule are fixed.

> **Choosing between models using the test scores turns the test into validation data.**

The pipeline is the unit of comparison

Every learned preparation step belongs inside cross-validation. [3]

Stage	During fit on a training fold	During validation
Imputation	Learn each feature's median	Reuse the training medians
Classifier	Learn splits from prepared rows	Predict prepared validation rows
Score	No validation labels used to fit	Compare predictions with true labels

> **Our numeric pipeline uses median imputation followed by the candidate classifier.**

Wine has no missing values. The imputer makes the workflow ready for missing numeric data.

How preprocessing can leak information

The validation fold must remain unseen while the pipeline learns.

Leakage before CV

Fit an imputer on all development rows, then run CV on the transformed data.

Validation rows helped determine the medians.

Preparation inside CV

For each fold, fit a fresh imputer on its training portion.

Transform its validation portion using only those learned medians.

> **The same rule applies to feature selection, scaling and other learned transforms.**

Five-fold cross-validation

Each candidate pipeline faces the same five versions of the experiment. [4]

Run	Fold A	Fold B	Fold C	Fold D	Fold E
1	Validate	Train	Train	Train	Train
2	Train	Validate	Train	Train	Train
3	Train	Train	Validate	Train	Train
4	Train	Train	Train	Validate	Train
5	Train	Train	Train	Train	Validate

Fit a fresh pipeline on four folds. Score it on the remaining fold. Repeat five times.

The 36 test rows never enter this table.

The split must match the prediction task

Data structure	Suitable starting point	Reason
Independent classification rows	StratifiedKFold	Keep class proportions similar
Repeated people or devices	GroupKFold or stratified group CV	Keep a group out of both sides
Predicting future observations	TimeSeriesSplit or temporal holdout	Training must precede validation

> **For this classroom example, we treat Wine rows as independent observations.**

Stratification balances labels. It does not remove dependence, duplicates or temporal leakage.

Grid search compares complete configurations

The best configuration is the one with the highest mean CV macro F1. [2]

Model	Candidate settings	Candidates
Decision Tree	3 depths x 2 leaf sizes	6
Random Forest	2 sizes x 3 depths x 2 leaf sizes x 2 feature rules	24

30 configurations x 5 folds = 150 fits

Then refit the selected pipeline once on all training rows.

This is an illustrative search budget, not a guarantee of the best possible model.

Code 1: data, split and pipeline

The four code slides form one script. Run them in order. Tested with scikit-learn 1.5.2.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("model", DecisionTreeClassifier(random_state=42))])
```

Code 2: a separate grid for each model

The model__ prefix addresses a setting of the pipeline step named model.

```
from sklearn.ensemble import RandomForestClassifier

grid = [
    {"model": [DecisionTreeClassifier(random_state=42)],
     "model__max_depth": [3, 6, None],
     "model__min_samples_leaf": [1, 4]},
    {"model": [RandomForestClassifier(random_state=42, n_jobs=1)],
     "model__n_estimators": [100, 300],
     "model__max_depth": [3, 6, None],
     "model__min_samples_leaf": [1, 4],
     "model__max_features": ["sqrt", 1.0]}
]
```

Separate dictionaries avoid assigning forest-only settings to the Decision Tree.

Code 3: search only the training data

A simple baseline checks whether the candidate models add useful predictive value.

```
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.dummy import DummyClassifier

baseline = cross_val_score(
    DummyClassifier(strategy="most_frequent"),
    X_train, y_train, cv=cv, scoring="f1_macro")
search = GridSearchCV(
    pipe, grid, scoring="f1_macro", cv=cv, n_jobs=1,
    refit=True, return_train_score=True)
search.fit(X_train, y_train)
print("Baseline:", baseline.mean())
print("Selected:", search.best_params_)
```

refit=True trains the winner on all 142 training rows after the search.

Code 4: inspect CV, then evaluate once

GridSearchCV calls the validation-fold scores test scores in cv_results_.

```
import numpy as np
from sklearn.metrics import f1_score, confusion_matrix

r = search.cv_results_
for kind in (DecisionTreeClassifier, RandomForestClassifier):
    ids = [i for i, p in enumerate(r["params"])
           if isinstance(p["model"], kind)]
    i = max(ids, key=lambda j: r["mean_test_score"][j])
    print(kind.__name__, r["params"][i])
    print(r["mean_test_score"][i], r["std_test_score"][i])
# Freeze the choice before opening the held-out test set.
pred = search.predict(X_test)
print("Test macro F1:", f1_score(y_test, pred, average="macro"))
print(confusion_matrix(y_test, pred))
```

mean_test_score above refers to CV validation, not the reserved X_test.

The measured comparison

Wine dataset, fixed split and five shared folds. Higher macro F1 is better.

Best candidate in each family	Mean CV F1	Fold SD	Mean train F1
Majority-class baseline	0.191	-	-
Decision Tree	0.905	0.057	1.000
Random Forest	0.979	0.029	0.997

Selected tree: depth 6, minimum leaf 1.

Selected forest: 300 trees, depth 3, minimum leaf 4, features sqrt.

These are selection scores. Searching many candidates can make the winning CV score optimistic.

The average is only part of the evidence

Compare each model on the same validation fold.



Random Forest

Decision Tree

The forest scores higher on all five shared folds.

> **Fold SD describes variation across folds. It is not a confidence interval.**

Final evaluation of the selected pipeline

The search selected the Random Forest before these test labels were used.

Actual / predicted	Class 0	Class 1	Class 2
Class 0	12	0	0
Class 1	0	14	0
Class 2	0	0	10

1.000 **Macro F1 on 36 held-out rows**

Rows are actual classes. Columns are predictions. This small test gives limited evidence.

A credible experiment report

What to record

Data version and split method.

Metric, folds, seeds and library versions.

Search space, chosen settings and final test result.

What to conclude

State what this experiment supports and where uncertainty remains.

If you repeatedly assess the whole tuning procedure, use nested CV with separate outer folds.

> **A fixed seed makes a run reproducible. It does not make one split representative.**

Your next experiment

Development exercise

01 Reproduce the comparison and explain why the forest won this run.

02 On training data only, test one justified change to the grid. Compare mean CV performance and fold variation.

03 Write a short model-choice argument that includes complexity and uncertainty, not just the largest score.

Once you have inspected this test set, do not use it as fresh evidence for later choices.

References and teaching materials

1 Random forests and class probabilities

scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

2 Grid search and model selection

scikit-learn.org/stable/modules/grid_search.html

3 Pipelines and leakage

scikit-learn.org/stable/common_pitfalls.html

4 Cross-validation and split strategies

scikit-learn.org/stable/modules/cross_validation.html

5 Wine dataset

scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html

6 Permutation feature importance

scikit-learn.org/stable/modules/permutation_importance.html

Course references

Dania Tamayo Vera, Bayesian Learning and ML Concepts (September 23, 2026).
Decision Trees lecture. Supplied four-slide Random Forest and ML Experimentation draft.

Numerical experiment executed with scikit-learn 1.5.2. Documentation accessed October 19, 2026.